

LeanBPE: A Formally Verified GPT-Family Byte-Level BPE Tokenizer for ASCII Text

FELIX WANG, University of Waterloo, Canada

Tokenizers are often treated as simple plumbing in LLM systems, but they decide what text the model actually sees [6]. In most deployments, this stage is expected to be lossless, yet that expectation is usually justified operationally rather than with machine-checked guarantees [5, 7]. This gap matters because tokenizer behavior can affect downstream capability and robustness [1, 2], and can also expose an interface for adversarial manipulation [8]. This paper addresses the gap with LeanBPE, a machine-checked formalization of a GPT-family byte-level BPE tokenizer for ASCII text in Lean 4. We formalize the core pipeline, including ASCII classification, pre-tokenization for `cl100k` and `o200k`, merge semantics, vocabulary construction, byte-shuffle inversion, and end-to-end roundtrip correctness. At load time, we check that the artifact contains all 256 base-byte tokens, that every merge target decodes to the concatenation of its parents in the constructed vocabulary, and that the bijective correspondence between serialized token IDs and their internal representations is preserved on the supported ASCII subset. Once these conditions hold, the formal results apply directly and guarantee that decoding the encoded output recovers the original ASCII input exactly. To evaluate practical behavior, we compare LeanBPE (Lean 4) with `tiktoken` (Rust-backed) and `MinBPE` (Python) on five ASCII-filtered domains covering software engineering, mathematics, chemistry, biomedical QA, and medicine. Although LeanBPE is slower than optimized implementations, making it less practical in deployment, it matches `tiktoken` on scoped correctness metrics with 100% agreement. The contribution of this project is thus a verified tokenizer core, an explicit validated-loader boundary for real tokenizer artifacts, and a reproducible empirical comparison framework that links formal guarantees to observed behavior. Within a practical deployment slice, this demonstrates that GPT-family byte-level BPE can be specified, validated, and machine-checked for lossless roundtrip behavior.

1 Introduction

Tokenization is the first boundary between user-facing text and model-facing computation. At that boundary, the token-level structure consumed by the model must remain semantically aligned with the text that users and developers are able to visually see. When this alignment fails, two artifacts that look identical can map to different byte sequences and therefore different model inputs. The impact is practical rather than cosmetic. Reproducibility degrades because prompt replay may not preserve the same effective input; debugging becomes harder because failures can arise from tokenization effects rather than model parameters; and safety enforcement becomes less reliable because many safeguards are expressed over rendered text while model behavior is driven by tokenized form [4, 6, 8]. This boundary also matters for evaluation and model comparison, since tokenizer design choices can influence both measured capability and robustness across tasks [1, 2]. For these reasons, tokenizer semantics should be treated as part of the trusted system interface and not as an implementation detail.

That boundary risk leads to a concrete technical requirement: for in-scope inputs, tokenization must be meaning-preserving end to end. In byte-level BPE pipelines [3, 10], the minimal form of that requirement is roundtrip fidelity, i.e., decoding the encoded sequence reproduces exactly the original string. This property is foundational because every later stage of training or inference depends on token IDs, not on rendered text. If roundtrip fidelity fails, the model is effectively conditioned on a transformed input, and the mismatch described above becomes a systematic source of error rather than an edge case. And in practice, production tokenizers such as `tiktoken` are engineered for speed and broad deployment [7]. They are highly effective operational systems, but their correctness is established primarily through implementation discipline and testing, not through machine-checked

proofs of end-to-end losslessness. At the same time, recent work increasingly treats tokenization as a first-order factor in capability and robustness [1, 2]. Together, these facts create a clear gap: tokenizer behavior is consequential, but its guarantees are often implicit.

This paper addresses that gap with LeanBPE, a Lean 4 formalization of a GPT-family byte-level BPE tokenizer for ASCII. We prove lossless encode/decode behavior under explicit structural assumptions, validate loaded tokenizer artifacts against those assumptions at runtime, and evaluate empirical agreement against tiktoken on multiple ASCII-filtered domains. The resulting contribution is not a faster tokenizer, but a clearer assurance story: a machine-checked semantic core, a documented trust boundary for real artifacts, and reproducible evidence that the verified implementation matches a widely used reference within the scoped setting.

2 Approaches

2.1 Overview of Byte-level BPE

Tokenization is the function that maps human-readable text into the discrete symbols consumed by a language model. In byte-level BPE, this map is defined over bytes and then compressed through learned merges, rather than being defined directly over words [3, 10]. This design avoids many language-specific tokenization choices and yields a deterministic interface from text to model input. For the GPT-family profiles, encoding is a composition of deterministic operators: UTF-8 byte conversion, profile-conditioned pre-tokenization, byte remapping (identity in the runtime configuration but abstract in the proof interface), and greedy merge reduction under a finite merge map. The decoding path inverts this composition by expanding token IDs through the vocabulary map, applying inverse remapping, concatenating chunk outputs, and decoding UTF-8. Our notion of correctness is therefore compositional: losslessness is not a property of any single stage in isolation, but of the full encode/decode composition.

2.2 Scope and Proof Target

The formal target is an ASCII-scoped roundtrip theorem:

$$\forall s \in \text{ASCII}, \text{decode}(\text{encode}(s)) = s.$$

The corresponding implementation-level endpoints are profile-aware internal-ID roundtrip theorems and an external-ID theorem for loaded artifacts. Crucially, the theorem is parameterized by explicit structural assumptions rather than hidden implementation facts: base-byte vocabulary soundness, merge decomposition soundness, and consistency of the external/internal ID translation on the reachable ID set.

Accordingly, we model and verify the tokenization core for ASCII under c1100k and o200k (two GPT-family tokenizer profiles with different merge/vocabulary configurations and pre-tokenization patterns), where semantic ambiguity from normalization and broader Unicode behavior is minimized. By design, we do not claim parser correctness from raw JSON bytes, full Unicode equivalence, or guarantees beyond tokenization itself. The methodological goal is to provide strong guarantees on a clearly bounded interface, with a transparent statement of what remains trusted but unchecked.

2.3 System Functionality

The system is organized around a proof-driven decomposition of the tokenizer pipeline. The first layer specifies byte-classification and pre-tokenization semantics for the two supported GPT-family profiles, together with correspondence results that tie executable splitting behavior to a declarative ASCII specification. The second layer formalizes the BPE merge dynamics and establishes the algebraic properties needed for composition, including decode preservation under valid merge steps and concatenation laws for decoded byte sequences.

The third layer captures construction-time invariants. A merge-list validity relation and associated construction lemmas define when generated merge and vocabulary maps are suitable for verified encoding. These construction lemmas are not merely bookkeeping: they provide the bridge from artifact structure to the exact proof obligations required by operational roundtrip theorems.

The final layer connects formal semantics to executable loading. Runtime loaders extract ASCII-compatible tokenizer artifacts, while boolean validators check the same obligations used by the proofs, including merge decomposition and ID-map consistency. Reflection theorems then lift successful validation into propositions, yielding a proof-carrying runtime object that can be consumed by the external-ID roundtrip theorem. This design makes the trust boundary explicit and auditable without collapsing the formal core into parser-specific details.

2.4 Theorem Construction

The theorem argument is organized to mirror the execution path of the tokenizer, moving from artifact loading to pre-tokenization, merge-based encoding, decode composition, and finally the external-ID roundtrip interface. The stages below summarize only the proof obligations that are essential for this end-to-end guarantee:

Load and certify artifact. The JSON-loading step itself is operational, but before any theorem is invoked the loaded artifact is certified against proof-relevant invariants. At this point, the development establishes a logical implication from successful runtime checks to formal assumptions: if validation succeeds, then the vocabulary decomposition constraints and ID-translation consistency conditions required by the proofs hold. This step turns an executable artifact into a proof-carrying object that can be used safely by later stages.

Pre-tokenization stage. At this stage, the executable pre-tokenizer is proved equivalent to the intended ASCII specification for each supported profile (c1100k and o200k). The development also proves that chunking is lossless at the byte level: concatenating the produced chunks recovers exactly the original UTF-8 byte stream.

Merge and decode stage. A single admissible merge step is proved decode-invariant, meaning that replacing a matched pair with its merged token preserves decoded bytes. Decoding is also proved compositional over concatenation, which allows local preservation arguments to lift from individual chunks to whole sequences. From these results, chunk-level preservation extends to full profile-aware encoding.

Decode-to-text stage. UTF-8 conversion is proved lossless on the ASCII domain considered in this work. Combining pre-tokenization correctness, merge/decode preservation, and UTF-8 roundtrip yields the internal-ID end-to-end roundtrip result.

External-ID interface stage. Finally, the internal-ID roundtrip guarantee is lifted to the user-facing API by proving that external/internal token-ID translation preserves the same roundtrip behavior.

Final composition. This composition is sufficient for the final goal because each stage discharges exactly one failure mode in the end-to-end path. Artifact certification rules out malformed runtime state; pre-tokenization theorems rule out boundary loss or profile mismatch; merge/decode theorems rule out semantic drift during compression; UTF-8 roundtrip rules out byte-to-string mismatch; and the external-ID theorem rules out translation inconsistency at the API boundary. Since the full tokenizer is precisely the composition of these stages, proving each stage and its interface condition yields the desired end-to-end guarantee: for validated artifacts and ASCII inputs, the user-facing encode/decode pipeline returns the original text.

3 Results

3.1 Motivations and Research Questions

We compare LeanBPE against two reference implementations with different design goals. `tiktoken` is OpenAI’s production tokenizer library, implemented with a Rust backend and optimized for high-throughput deployment [7]. `MinBPE` is a compact educational Python implementation of GPT-style byte-level BPE that prioritizes readability and conceptual transparency over systems optimization. Using both baselines helps separate two effects in our measurements: the gap between a verified implementation and an optimized production system, and the gap between a verified implementation and a simple, interpretable reference in a high-level language.

The evaluation focuses on three tokenizer characteristics that matter most for practical use: throughput, memory footprint, and correctness. First, tokenization sits directly on the critical path of both training and inference pipelines, so throughput is a primary systems concern. Prior work on tokenizer implementation has explicitly treated speed as a core objective and shown large gains from algorithmic improvements [5, 11].

RQ1: How does LeanBPE throughput compare with `tiktoken` and `MinBPE` across domains and tokenizer profiles?

Second, tokenizer choice affects sequence length and operational state size, which in turn influences downstream compute and memory behavior. Recent work emphasizes that tokenization design cannot be reduced to simple compression and has non-trivial system-level consequences [9]; related byte-level modeling studies also report explicit trade-offs among sequence length, compute, and efficiency [13].

RQ2: What memory footprint does LeanBPE exhibit relative to the baselines under `c1100k` and `o200k` on the same workloads?

Third, because this project is centered on a verified losslessness claim, empirical correctness must be measured explicitly in addition to theorem statements. Prior analyses show that tokenizer design materially affects model behavior and robustness [1, 2], motivating strict agreement checks against a widely used reference implementation.

RQ3: On ASCII-scoped inputs, does LeanBPE achieve full roundtrip success and full encode/decode agreement with `tiktoken`?

To answer these questions, all experiments use ASCII-filtered corpora from five domains: **software engineering** (SWE-Bench), **mathematical reasoning** (GSM8K), **chemistry** (ChemistryQA), **biomedical QA** (BioASQ), and **medicine** (MedMCQA). Speed and memory metrics are reported as means over five full benchmark runs, and correctness is reported from per-run differential studies against `tiktoken`.

3.2 RQ1–RQ2: Speed and Memory

RQ1: How does LeanBPE throughput compare with `tiktoken` and `MinBPE`? Table 1 shows a consistent throughput ordering across domains: `tiktoken` is the fastest, `MinBPE` is intermediate, and LeanBPE is the slowest. Averaged over all domain/profile settings, mean throughput is 2.25M tokens/s for `tiktoken`, 243k tokens/s for `MinBPE`, and 171k tokens/s for LeanBPE. This corresponds to an approximately $13.1\times$ gap between `tiktoken` and LeanBPE, and a $1.42\times$ gap between `MinBPE` and LeanBPE. To test whether these differences are systematic, we fit an ANOVA model on

Table 1. Benchmark by Domain (mean of 5 runs)

Engine	Language	Tokenizer	Num Tokens	Total Time (s)	Tokens/s	Peak RSS (MiB)
Software Engineering (SWE-Bench)						
tiktoken	Rust	cl100k	8,160,343	3.459,314	2,359,985.95	123.8
tiktoken	Rust	o200k	8,229,480	3.464,392	2,375,685.50	344.1
MinBPE	Python	cl100k	8,160,343	27.732,831	294,262.98	130.4
MinBPE	Python	o200k	8,229,480	28.710,143	286,696.84	361.7
LeanBPE	Lean 4	cl100k	8,160,343	153.220,907	53,262.25	172.0
LeanBPE	Lean 4	o200k	8,229,480	161.680,742	50,902.72	290.4
Mathematical Reasoning (GSM8K)						
tiktoken	Rust	cl100k	913,466	0.406,611	2,270,784.60	100.0
tiktoken	Rust	o200k	905,214	0.398,652	2,273,923.60	342.9
MinBPE	Python	cl100k	913,466	3.762,988	243,705.62	100.5
MinBPE	Python	o200k	905,214	3.667,930	246,818.83	351.7
LeanBPE	Lean 4	cl100k	913,466	4.319,790	211,527.70	139.9
LeanBPE	Lean 4	o200k	905,214	5.451,300	166,114.49	255.9
Chemistry (ChemistryQA)						
tiktoken	Rust	cl100k	79,301	0.032,526	2,438,738.58	98.2
tiktoken	Rust	o200k	79,010	0.033,924	2,329,404.20	344.5
MinBPE	Python	cl100k	79,301	0.273,084	300,775.57	95.9
MinBPE	Python	o200k	79,010	0.253,331	311,909.81	347.9
LeanBPE	Lean 4	cl100k	79,301	0.296,335	267,698.53	135.3
LeanBPE	Lean 4	o200k	79,010	0.392,900	201,619.61	254.5
Biology / Biomedical QA (BioASQ)						
tiktoken	Rust	cl100k	48,072	0.021,969	2,188,953.24	103.0
tiktoken	Rust	o200k	47,002	0.023,947	1,964,002.16	344.7
MinBPE	Python	cl100k	48,072	0.248,191	193,735.52	98.2
MinBPE	Python	o200k	47,002	0.255,309	184,151.04	346.2
LeanBPE	Lean 4	cl100k	48,072	0.217,815	220,827.52	135.2
LeanBPE	Lean 4	o200k	47,002	0.273,134	172,139.36	254.5
Medicine (MedMCQA)						
tiktoken	Rust	cl100k	3,128,322	1.423,548	2,199,559.34	105.6
tiktoken	Rust	o200k	3,054,188	1.491,667	2,056,519.63	344.1
MinBPE	Python	cl100k	3,128,322	16.750,102	187,434.37	120.5
MinBPE	Python	o200k	3,054,188	16.551,811	184,549.01	351.4
LeanBPE	Lean 4	cl100k	3,128,322	14.995,996	208,640.92	161.4
LeanBPE	Lean 4	o200k	3,054,188	18.940,461	161,355.41	280.9

$\log(\text{tokens/s})$ with factors engine, tokenizer family, and dataset. The engine effect is strongly significant ($F = 173.7$, $p = 3.34 \times 10^{-14}$), while tokenizer family and dataset are not significant after controlling for engine ($p = 0.489$ and $p = 0.165$). A separate regression of total tokenization time on token count within each engine also shows near-linear scaling (LeanBPE $R^2 = 0.928$, MinBPE

$R^2 = 0.955$, tiktoken $R^2 = 0.998$), indicating that the observed ranking is stable across workload size rather than driven by a single dataset.

RQ2: What memory footprint does LeanBPE exhibit relative to the baselines? The memory trend differs from the speed trend: peak RSS (Resident Set Size) is driven primarily by tokenizer family rather than by implementation engine. ANOVA on $\log(\text{RSS MiB})$ again with engine, tokenizer family, and dataset shows a dominant tokenizer-family effect ($F = 247.2$, $p = 1.90 \times 10^{-13}$), while engine and dataset are not significant ($p = 0.867$ and $p = 0.449$). Concretely, o200k uses substantially more memory than c1100k; the geometric mean ratio is $2.67\times$ across all engine-dataset pairs. By engine, the mean o200k/c1100k RSS ratio is $1.80\times$ for LeanBPE, $3.27\times$ for MinBPE, and $3.26\times$ for tiktoken. Thus, for memory, the major explanatory variable is tokenizer profile size rather than language/runtime choice.

3.3 RQ3: Correctness and Agreement

Table 2 reports output accuracy using tiktoken as the reference tokenizer. Across both supported tokenizers and all five domains, LeanBPE achieves 100% in all three metrics: roundtrip passes, encode agreement with tiktoken, and decode-on-reference-IDs passes. Aggregated over runs, this corresponds to 435,130/435,130 successes for each metric, with zero observed failures.

These results show that, on the evaluated ASCII datasets, tiktoken’s c1100k and o200k tokenization behavior is fully lossless and in exact agreement with LeanBPE.

Table 2. Output accuracy using tiktoken as a reference

Tokenizer	Dataset	Roundtrip Passes	Encode Agreement	Decode(Python IDs) Passes	Rate
c1100k/o200k	BioASQ	3632/3632	3632/3632	3632/3632	100.00%
c1100k/o200k	ChemistryQA	2474/2474	2474/2474	2474/2474	100.00%
c1100k/o200k	GSM8K	16033/16033	16033/16033	16033/16033	100.00%
c1100k/o200k	MedMCQA	179234/179234	179234/179234	179234/179234	100.00%
c1100k/o200k	SWE_Bench	16192/16192	16192/16192	16192/16192	100.00%

Section 3 Summary.

What was achieved: LeanBPE matches tiktoken with 100% output agreement on the evaluated ASCII datasets and provides a theorem-backed lossless pipeline under validated runtime assumptions.

What was not achieved: Support for non-ASCII characters, throughput competitive with optimized production implementations, full Unicode/ $\backslash p\{\}$ semantics, and end-to-end formal verification of raw JSON parsing/extraction.

4 Discussion

4.1 Interesting Aspects

The most interesting aspect of the project was the pre-tokenizer design tradeoff between a hand-written FSM and a formally verified regex engine. The initial FSM route worked semantically, but in practice it was slow enough to be barely usable and difficult to evolve as profile rules became more complex. Migrating to a formally verified regex-based layer gave a cleaner specification interface and a more maintainable proof structure, while avoiding the operational brittleness of the FSM implementation. The key compromise was expressive coverage: because the verified regex stack did not provide full support for Unicode property classes such as $\backslash p\{L\}$ and $\backslash p\{N\}$, those classes were replaced with explicit ASCII-compatible predicates. This was an intentional scope decision:

we gave up full Unicode-level pattern fidelity in exchange for a simpler, verifiable, and practically usable ASCII pipeline.

4.2 Design Decisions

The design decisions across the project were feedback-informed and result-driven, with scope refined iteratively as proofs and experiments revealed new constraints. The original scope focused on core BPE operations under a global well-formedness assumption, without targeting a standalone, artifact-driven package. As the project progressed, the goal shifted toward an end-to-end usable system, which required reducing implicit assumptions and exposing them as explicit, checkable conditions. This change directly motivated the validated-loader interface and the decision to support loading multiple HuggingFace-style GPT-family `tokenizer.json` artifacts rather than proving only a single fixed configuration.

A second design story concerns how we handled assumptions. Early proofs were clean only because they started from a strong global well-formedness hypothesis, which was mathematically convenient but weak as an engineering interface. In practical use, users do not hand us a theorem; they hand us a `tokenizer.json`. We therefore changed the interface so that loading and validation come first, and theorem application comes second. Concretely, the loader now checks the structural obligations that the proofs actually need (base-byte coverage, merge decomposition, and ID-map consistency), and only then exposes a certified runtime object to the verified encode/decode path. This decision traded stronger static elegance for a more realistic trust model, but it made the package substantially more usable without weakening the formal claim inside the declared scope.

4.3 Challenges

The hardest challenge was not BPE itself, but repeatedly deciding what should be in scope at each stage of development. The first concrete tradeoff appeared in the FSM-to-regex migration: keeping the FSM preserved direct control over semantics but made the system too slow to be practically usable; moving to a verified regex engine improved maintainability and performance, but required sacrificing full Unicode property coverage and committing to ASCII substitutions. The second tradeoff appeared in artifact loading: fully verifying JSON parsing and extraction would have enlarged the trusted proof surface beyond feasible project limits, so we instead verified the semantic obligations needed downstream and proved that successful runtime validation implies those obligations.

Another challenge was ensuring that formalization fidelity matched the intended tiktoken behavior, not merely that Lean produced successful proof checks. In early iterations, some implementations were internally consistent yet still disagreed with reference behavior on concrete edge patterns (for example, profile-specific pre-tokenization boundaries and ID translation details). This forced a continuous loop of theorem refinement, loader/interface redesign, and differential testing against tiktoken. The central lesson was that “proof complete” and “specification faithful” are different milestones, and both must be met for a credible tokenizer verification claim.

5 Reflection

The most important lesson is that better automation does not remove verification work; it changes where the bottleneck is. As autoformalization improves [12], routine implementation checks can be discharged faster, but the hardest problem becomes requirement fidelity: whether the formal statement actually captures what the tokenizer is supposed to do. In our case, many components could be made internally consistent, yet that alone did not guarantee alignment with the intended GPT-family behavior. From this perspective, software testing in the future is likely to shift from predominantly behavior-sampling tests toward specification-centric validation of assumptions,

interfaces, and invariants. This suggests a future workflow where testing does not disappear, but is reallocated toward validating specifications, interfaces, and assumptions rather than only exercising code paths.

A second lesson is about sycophancy in code generation. The risk is highest during the formalization phase, where requirements are still being translated from natural language and a model can overfit to user phrasing while missing implicit constraints. I saw this pattern in early iterations: changes could look locally reasonable yet drift from the intended semantics. After formalization stabilizes, the implementation phase is no longer like ordinary code generation in mainstream languages, because Lean supplies a self-supervisable oracle through proof obligations and type checking. Once that oracle is in place, both humans and coding agents can iterate against concrete proof feedback rather than conversational plausibility alone.

6 Future Work

Future work should focus on closing the remaining assurance gap and improving practical deployment value. The most important extension is parser and extractor verification from raw JSON into certified internal representations. A second extension is Unicode-general reasoning, including normalization-aware semantics beyond the current ASCII scope. On the systems side, concrete Lean optimizations include replacing hot `List`-based encode/decode loops with array-backed traversals, reducing intermediate `ByteArray` allocations in pre-tokenization and merge application, and caching frequently accessed merge/vocabulary lookups during chunk encoding. A concrete hybrid path is to keep Lean as the proof authority while offloading throughput-critical kernels (e.g., pre-tokenizer scanning and merge-loop execution) to a Rust runtime module, then checking the returned outputs against Lean-side invariants before exposing results at the API boundary. Finally, larger differential testing campaigns, including adversarial and randomized corpora, would further strengthen empirical confidence and help prioritize optimization work. Collectively, these directions would move LeanBPE from a strong verification prototype toward a broadly deployable, high-assurance tokenizer framework.

References

- [1] Gül Sena Altıntaş, Malikeh Ehghaghi, Brian Lester, Fengyuan Liu, Wanru Zhao, Marco Ciccone, and Colin Raffel. 2025. TokSuite: Measuring the Impact of Tokenizer Choice on Language Model Behavior. *arXiv preprint arXiv:2512.20757* (2025).
- [2] Navid Ayoobi, Marcus I Armstrong, and Arjun Mukherjee. 2026. Say Anything but This: When Tokenizer Betrays Reasoning in LLMs. *arXiv preprint arXiv:2601.14658* (2026).
- [3] Philip Gage. 1994. A new algorithm for data compression. *C Users J.* 12, 2 (Feb. 1994), 23–38.
- [4] Juan Luis Gastaldi, John Terilla, Luca Malagutti, Brian DuSell, Tim Vieira, and Ryan Cotterell. 2025. The Foundations of Tokenization: Statistical and Computational Concerns. In *The Thirteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=B5iOSxM2I0>
- [5] Taku Kudo and John Richardson. 2018. SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the 2018 conference on empirical methods in natural language processing: System demonstrations*. 66–71.
- [6] Sabrina J Mielke, Zaid Alyafeai, Elizabeth Salesky, Colin Raffel, Manan Dey, Matthias Gallé, Arun Raja, Chenglei Si, Wilson Y Lee, Benoît Sagot, et al. 2021. Between words and characters: A brief history of open-vocabulary modeling and tokenization in nlp. *arXiv preprint arXiv:2112.10508* (2021).
- [7] Openai. [n. d.]. Openai/tiktoken: Tiktoken is a fast BPE tokenizer for use with OpenAI’s models. <https://github.com/openai/tiktoken>
- [8] Ephraim Sarabamoun. 2025. Special-Character Adversarial Attacks on Open-Source Language Model. *arXiv preprint arXiv:2508.14070* (2025).
- [9] Craig W Schmidt, Varshini Reddy, Haoran Zhang, Alec Alameddine, Omri Uzan, Yuval Pinter, and Chris Tanner. 2024. Tokenization is more than compression. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 678–702.

- [10] Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016. Neural Machine Translation of Rare Words with Subword Units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Katrin Erk and Noah A. Smith (Eds.). Association for Computational Linguistics, Berlin, Germany, 1715–1725. doi:10.18653/v1/P16-1162
- [11] Xinying Song, Alex Salcianu, Yang Song, Dave Dopson, and Denny Zhou. 2021. Fast wordpiece tokenization. In *Proceedings of the 2021 conference on empirical methods in natural language processing*. 2089–2103.
- [12] Yuhuai Wu, Albert Qiaoju Jiang, Wenda Li, Markus Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. 2022. Autoformalization with large language models. *Advances in neural information processing systems* 35 (2022), 32353–32368.
- [13] Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. 2022. ByT5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics* 10 (2022), 291–306.

A Appendix

A.1 Use of LLMs

I have used an LLM-based writing assistant to polish grammar. All ideas, analyses, experiments, and scientific claims are my own, and I take full responsibility for the content of this work.

A.2 Replication Packgae

Codebase and replication package could be found at: <https://github.com/U70-TK/BPETokenizer>.

A.3 Tables & Diagrams

Table 3. LLM Inference Pipeline and the Tokenization Boundary

Stage	Pseudo-Code	Description
Raw Input	$s \in \Sigma^*$	Unicode input string provided by user.
Normalization	$\tilde{s} \leftarrow \text{Normalize}(s)$	Optional preprocessing (e.g., NFKC).
Tokenization	$t \leftarrow \text{encode}(\tilde{s})$	Map string into discrete token IDs in vocabulary $\mathcal{V}$.
Embedding	$x_i^{(1)} \leftarrow \text{Embed}(t_i)$	Convert each token ID into a vector in $\mathbb{R}^{d_{\text{model}}}$.
Self-Attention	$\tilde{x}_i^{(l)} \leftarrow x_i^{(l)} + \text{Attn}^{(l)}(x_{1:i}^{(l)})$	Contextual transformation across tokens.
Feed-Forward	$x_i^{(l+1)} \leftarrow \tilde{x}_i^{(l)} + \text{MLP}^{(l)}(\tilde{x}_i^{(l)})$	Position-wise nonlinear transformation.
Output Projection	$\text{logits}_i \leftarrow \text{Unembed}(x_i^{(L)})$	Map final hidden state back to vocabulary space.
Softmax	$y_i \leftarrow \text{softmax}(\text{logits}_i)$	Probability distribution over next tokens.
Detokenization	$\hat{s} \leftarrow \text{decode}(t)$	Recover text from token sequence.
Desired Property: $\text{decode}(\text{encode}(\tilde{s})) = \tilde{s}$		